TP Integrador 1 – Teoría para el coloquio

Redes de Datos II · LINTI–UNLP · Grupo 0P · Bloques asignados: **130.0.16.0/20** y **2002:dae1::/32**

Todo lo que se usó en los puntos obligatorios del TP, explicado desde cero: definición → regla → cómo se hace → ejemplo con nuestra red. Basado en las clases de la cátedra (IPv4, IPv6, ARP, ICMP) y en los ejemplos resueltos de la práctica. Archivo de la resolución: `tpi_1_nuevo.xml` (versión final, emulador CORE). Cada punto termina con un recuadro verde "Cómo defenderlo".

Contenido

1. Antes de empezar: qué es cada cosa
2. El TP en una página
3. Direccionamiento IPv4 (VLSM)
4. Punto a: IPv6 en Net5 y Net6
5. Punto b: ruteo estático IPv4 e IPv6
6. Punto d: ping y traceroute
7. Punto e: captura n11 → n33 (ARP e ICMP)
8. Punto f: traceroute capturado
9. Punto h: IPv6 e ICMPv6
10. CORE y comandos
11. Errores conceptuales que hay que evitar

Cada punto tiene arriba un recuadro con lo que pide el enunciado y lo que tenés que saber explicar; al final, un recuadro verde con cómo defenderlo y las preguntas probables.

1. Antes de empezar: qué es cada cosa

Esta sección no tiene nada del TP todavía. Explica con palabras simples los equipos y las ideas que aparecen en todo el resto. Si entendés esta página, el resto se lee mucho más fácil.

Los equipos

Equipo	Qué es	En el TP
Host	Una computadora "final": la que usa una persona o un servidor. Manda y recibe sus propios paquetes, pero no reenvía paquetes de otros.	n11, n12, n13, n14, n29 … n35 (los íconos de PC del dibujo)
Switch	Conecta varios equipos dentro de **una misma red local** (LAN). No mira direcciones IP; solo reparte tramas por MAC. No sabe nada de ruteo.	Los íconos chatos debajo de cada router de acceso (n3, n4, n19…)
Router	Conecta **redes distintas** entre sí. Mira la IP de destino de cada paquete y decide por qué cable mandarlo. Tiene una IP distinta en cada cable (interfaz).	Todos los íconos redondos: spines, leafs y routers de acceso
Router de acceso	El router "de la puerta" de cada LAN. Es la salida de esa red hacia el resto del mundo.	n1, n2, n17, n18, n21, n22, n23, n24
Leaf (hoja)	Router de nivel intermedio. Junta a varios routers de acceso y los conecta con el núcleo. Se llama "hoja" porque está en el borde del árbol, del lado de las redes de usuarios.	leaf1, leaf2, leaf3, leaf4 (uno por cada grupo Net1 … Net4)
Spine (columna)	Router del **núcleo**. Une a todos los leafs entre sí. Se llama "columna vertebral" porque todo el tráfico entre leafs pasa por él.	spine1 y spine2

La idea de spine-leaf, en simple

Pensalo como un edificio de oficinas. Cada piso tiene su **ascensor de piso** (leaf) que junta a las oficinas de ese piso (las redes Net1, Net2…). Los pisos no están conectados directamente entre sí. Para ir de un piso a otro siempre se pasa por el **hall central** (spine). Hay dos halls centrales para que, si uno se rompe, quede el otro.

Nivel 1 · Spines (núcleo): spine1 y spine2. Conectan a todos los leafs. No se conectan entre sí. De spine1 cuelga Net5 y de spine2 cuelga Net6.

▲ ▼

Nivel 2 · Leafs (distribución): leaf1 a leaf4. Cada uno conectado a los DOS spines y a sus dos routers de acceso. No se conectan entre sí.

▲ ▼

Nivel 3 · Routers de acceso: n1, n2, n17, n18, n21, n22, n23, n24. Cada uno es la puerta de salida de una LAN.

▲ ▼

Nivel 4 · LANs (switch + hosts): las redes de usuarios. Net1 a Net4 tienen dos LANs cada una; Net5 y Net6 cuelgan directo de los spines.

Figura A. Los cuatro niveles de la topología, de arriba (núcleo) hacia abajo (usuarios).

Para ir de un host de Net1 a uno de Net4, el paquete sube y baja por la escalera: **host → router de acceso → leaf1 → spine → leaf4 → router de acceso → host**. Nunca hay atajos entre leafs ni entre spines.

Ideas que se repiten en todo el TP

- **Red (o subred) / LAN:** un grupo de equipos que se pueden hablar directamente, sin pasar por un router. Todos tienen IPs que empiezan igual (el mismo "prefijo de red").
- **Interfaz:** cada "puerto" o placa de red de un equipo (eth0, eth1…). Cada interfaz tiene su propia IP. Un router con 4 cables tiene 4 IPs.
- **Enlace punto a punto:** un cable que une exactamente dos routers, sin switch. Como solo hay dos equipos, se le da una red diminuta de 2 IPs (/30).
- **Gateway (puerta de enlace):** el router al que un host le manda todo lo que no es de su propia red. Para n11 es n1 (130.0.16.1).
- **Dirección IP vs dirección MAC:** la IP dice **a quién va el paquete al final** (como la dirección escrita en un sobre). La MAC dice **a quién se lo entrego en este tramo** (como el cartero que lo pasa de mano en mano). La IP queda igual todo el viaje; la MAC cambia en cada salto.
- **Tabla de ruteo:** la lista de carteles que tiene cada router: "para ir a tal red, mandalo a tal vecino". El router la consulta para cada paquete.
- **Ruta por defecto (default):** el cartel "para cualquier otro lado, por acá". Se usa cuando ningún otro cartel coincide.
- **Next-hop (próximo salto):** la IP del vecino al que se le pasa el paquete. Tiene que estar conectado directamente al router.
- **Ruteo estático:** las rutas las escribe una persona a mano (es lo que hizo el grupo). Lo opuesto es el ruteo dinámico, donde los routers se las enseñan solos con un protocolo.
- **Dual stack:** un equipo tiene a la vez una IP versión 4 (130.0…) y una versión 6 (2002:dae1…), y las dos funcionan por separado. En este TP solo los equipos del camino entre Net5 y Net6 tienen las dos.

> **Cómo viaja un paquete, en cuatro frases**
>
> 1) El host mira si el destino está en su propia red; si no, se lo da a su gateway.
> 2) Cada router mira la IP de destino, busca en su tabla el cartel que mejor coincide y se lo pasa al vecino que indica.
> 3) En cada tramo, el paquete viaja dentro de una "trama" con las MAC de ese tramo; para saber la MAC del vecino se usa ARP.
> 4) El último router ve que el destino está en una red suya y se lo entrega directo. La respuesta hace el mismo proceso al revés.

> **Cómo leer las IPs abreviadas de este documento**
>
> Cuando dice ".26.2" quiere decir 130.0.26.2: se omite el comienzo "130.0." porque es igual en todo el TP.

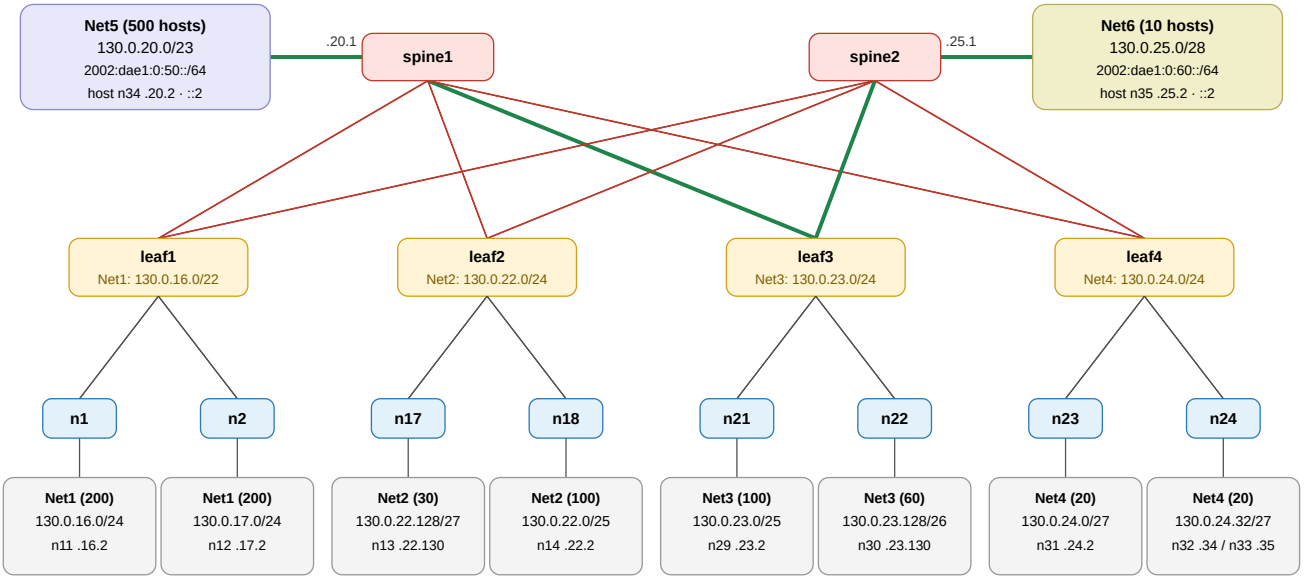
2. El TP en una página

> **Si ves números como /22, /24, /27 y no sabés qué son, no te preocupes**
> En esta página solo está el resumen. De dónde sale cada número se explica paso a paso en la sección 3 (Direccionamiento IPv4).

Qué pide el enunciado. Sobre la topología de la Figura 6 (dos spines, cuatro leafs, ocho routers de acceso y seis grupos de redes), direccionar con IPv4 usando el bloque asignado, con enlaces punto a punto /30 o /31 salvo donde hay switches. Después:

Punto	Qué pide	Estado en el archivo final
—	Direccionamiento IPv4 con VLSM	Hecho y verificado. Sección 3
a	IPv6 en Net5 y Net6, con conectividad IPv6 entre ambas	Hecho: IPv6 en Net5, Net6 y el camino entre ellas (spine1 – leaf3 – spine2). Sección 4
b	Ruteo estático IPv4 e IPv6	Hecho y verificado: IPv4 en toda la red, IPv6 en el camino Net5 – Net6. Sección 5
d	Probar con ping (ICMP) y traceroute	Hay que ejecutarlo en CORE. Sección 6
e	Capturar tráfico n11 → n33 y analizar ARP e ICMP	Hay que ejecutarlo en CORE. Sección 7
f	traceroute n11 → n33 capturando los mensajes	Hay que ejecutarlo en CORE. Sección 8
h	Probar conectividad IPv6 por separado y analizar ICMPv6	Hay que ejecutarlo en CORE. Sección 9

Los puntos c (NAT) y g (MTU) son opcionales, no se hicieron y esta guía no los cubre. Las pruebas (d, e, f, h) no quedan guardadas en el archivo: se muestran en vivo o con capturas de pantalla.



Rojo: enlaces leaf–spine (/30, todos en 130.0.26.0/24). Verde grueso: el camino que además tiene IPv6 (Net5 – spine1 – leaf3 – spine2 – Net6).
Gris: enlaces acceso–leaf (/30, dentro del bloque de cada Net). Gateway de cada LAN: la primera IP usable.
Cada leaf: sus dos LANs vía los routers de acceso · Net6 vía spine2 · todo lo demás (default) vía spine1.

Figura 1. Topología con el direccionamiento del archivo final (`tpi_1_nuevo.xml`). Dentro de cada leaf, el bloque de su Net.

Si estudiaste la versión anterior de esta guía

El archivo final cambió dos cosas: (1) **todas las direcciones IPv4**: ahora cada Net tiene su propio bloque (Net1 en 130.0.16.0/22, Net5 en 130.0.20.0/23, Net2 en 130.0.22.0/24, Net3 en 130.0.23.0/24, Net4 en 130.0.24.0/24, Net6 en 130.0.25.0/24 y los enlaces leaf–spine en 130.0.26.0/24); (2) **IPv6 ya no está en toda la red**, solo en Net5, Net6 y el camino entre ellas. Lo teórico (qué es una máscara, ARP, ICMP, NDP) no cambió.

Cada punto en una frase (para arrancar la defensa)

Punto	Qué decir
Direccionamiento	"Calculamos el tamaño de cada red con $2^h - 2$, le dimos a cada Net un bloque propio y alineado, y adentro repartimos con VLSM de mayor a menor. Enlaces /30."
a	"Net5 y Net6 tienen un /64 cada una. Como los spines no se tocan, el IPv6 pasa por leaf3, cuyos dos enlaces con los spines también tienen un /64."
b	"Hosts y routers de acceso: ruta por defecto hacia arriba. Leafs: sus dos LANs, Net6 por spine2 y el resto por spine1. Spines: una ruta sumarizada por Net, sin default."
d	"Probamos con ping entre redes distintas y traceroute: si responde, el direccionamiento y las rutas de ida y vuelta están bien."
e	"n11 hace ARP por su gateway, no por n33. En cada salto cambian las MAC; las IP no."
f	"traceroute manda UDP con TTL 1, 2, 3…; cada router responde Time Exceeded y n33 responde Port Unreachable."
h	"En IPv6 no hay ARP: se usa NDP (Neighbor Solicitation a una multicast y Neighbor Advertisement), todo dentro de ICMPv6."

Al final de cada punto hay un recuadro verde **"Cómo defenderlo"** con el guion completo y qué mostrar en CORE.

> **Trampa al leer el archivo**
>
> En el XML cada equipo tiene un número interno (`device id="1"`) y un nombre (`name="n34"`). El que vale es el **nombre**, que es el que se ve en el dibujo. Ejemplo: `device id="1"` es el host n34 de Net5; `device id="22"` es n5-spine1. En los `link`, `node1` y `node2` usan el número interno.

3. Direccionamiento IPv4 (VLSM)

Qué pide el enunciado: "Para la figura 6, resolver el direccionamiento IPv4 con el bloque IP asignado. Considerar los enlaces punto a punto /30 o /31, salvo las redes donde hay switches/hubs." Net1: dos redes de 200 hosts · Net2: una de 30 y otra de 100 · Net3: una de 100 y otra de 60 · Net4: dos de 20 · Net5: una de 500 · Net6: una de 10.

Qué hizo el grupo / qué hay que mostrar: Hecho. Se repartió el bloque 130.0.16.0/20 dándole a cada Net un bloque propio y, adentro de cada uno, VLSM. Los 16 enlaces entre routers son /30.

Lo que tenés que poder explicar:

- Qué es una IP, una máscara y qué significa la barra (/24, /27).
- Cómo se calcula el tamaño de cada red y por qué se reparte de mayor a menor.
- Por qué cada Net tiene su propio bloque (y por qué el de Net1 es un /22).
- La dirección de red, el broadcast y el rango usable de cualquier red del TP.
- Por qué los enlaces entre routers son /30 (y qué sería un /31).

3.1 IP: lo básico

> **Protocolo IP (RFC 791)**
> Protocolo de la capa de red. Es **no orientado a conexión** y de **mejor esfuerzo** (best effort): no garantiza entrega, orden ni ausencia de errores en los datos. Su unidad de datos (PDU) es el **datagrama** o paquete. Sus funciones: direccionamiento, ruteo/forwarding, multiplexación de protocolos superiores y fragmentación.

Capa de enlace vs capa de red. La capa de enlace (Ethernet, 802.11) lleva una **trama** entre dos nodos vecinos del mismo enlace, usando direcciones **MAC** (físicas, 48 bits). La capa de red lleva el **paquete** de origen a destino a través de muchas redes, usando direcciones **IP** (lógicas). Un paquete IP viaja dentro de una trama distinta en cada salto.

Campos del header IPv4 que hay que conocer

El header ocupa entre 20 y 60 bytes (20 sin opciones).

Campo	Bits	Para qué sirve
Version	4	4 para IPv4
Header Length (IHL)	4	Largo del header en palabras de 32 bits (5 = 20 bytes)
Total Length	16	Largo total del datagrama (header + datos), en bytes. Máximo 65.535
Identification	16	Igual en todos los fragmentos del mismo datagrama original
Flags	3	Reservado (0), **DF** (Don't Fragment), **MF** (More Fragments)
Fragment Offset	13	Posición del fragmento en el original, en unidades de 8 bytes
TTL	8	Se decrementa en 1 en cada router. Si llega a 0, se descarta. Evita loops
Protocol	8	Qué lleva adentro: 1 = ICMP, 6 = TCP, 17 = UDP
Header Checksum	16	Control de errores del header. Se recalcula en cada router (porque cambia el TTL)
Source / Destination Address	32 c/u	IP origen y destino

3.2 Direcciones, máscara y prefijo

Primero, la idea general: bloque y subredes

1) La cátedra le dio al grupo un **bloque**: 130.0.16.0/20. Es un "terreno" de 4096 direcciones (de 130.0.16.0 a 130.0.31.255). **Ningún equipo usa /20**: es solo el espacio que tenemos para repartir.
2) Ese bloque se **reparte** en redes más chicas (**subredes**), una por cada LAN y una por cada enlace entre routers. Cada subred tiene su propia barra (/23, /24, /27…) según cuántos equipos necesita. Cómo se hizo el reparto está en el apartado 3.3.
3) Cada equipo usa la IP y la **barra de la subred donde está conectado**. Por ejemplo, n33 está en una LAN a la que le tocó una subred /27, por eso su IP se escribe 130.0.24.35**/27**.
En esta sección vemos qué significa la barra y cómo se calcula el rango de una subred. En el apartado 3.3, cómo se eligió cada una.

Dirección IP

Número de 32 bits que identifica una **interfaz** (no un equipo). Por eso un router tiene una IP por cada interfaz. Se escribe en notación decimal con puntos: 130.0.24.35.

Máscara de red / prefijo

Indica cuántos bits de la izquierda son la parte de **red**; el resto es la parte de **host**. Se escribe como máscara (255.255.255.224) o como largo de prefijo (/27). Son lo mismo: /27 = 27 unos seguidos de 5 ceros.

En criollo: qué significa la barra

Una IP son 32 bits (cuatro números de 0 a 255, cada uno de 8 bits). El número después de la barra dice cuántos de esos bits, contando desde la izquierda, son el "nombre de la red"; los que sobran numeran a los equipos dentro de esa red.

· **/24**: los tres primeros números son la red y el último numera equipos. 130.0.16.0/24 = todas las IPs 130.0.16.*algo*.

· **/23**: la red es un poco más grande: abarca dos "terceros números" seguidos (130.0.20.*x* y 130.0.21.*x*).

· **/22**: abarca cuatro terceros números seguidos (130.0.16.*x* a 130.0.19.*x*).

· **/27**: la red es más chica: solo 32 IPs del último número (por ejemplo de .32 a .63).

Cuanto **más grande** el número de la barra, **más chica** la red.

Regla formal: la red a la que pertenece una IP se obtiene con un **AND** bit a bit entre la IP y la máscara.

Direcciones de una subred

- **Dirección de red:** todos los bits de host en 0. No se asigna a nadie.
- **Broadcast dirigido:** todos los bits de host en 1. No se asigna a nadie.
- **Usables:** las del medio. Cantidad = $\mathbf{2^h - 2}$, con h = bits de host = 32 − prefijo.

Ejemplo con nuestra red: n33 = 130.0.24.35/27

Dato de partida: n33 está en la LAN de n24, y en el reparto (apartado 3.3) a esa LAN le tocó una subred /27. Ahora calculamos qué rango es:

/27 → 32 − 27 = 5 bits de host → bloques de 2^5 = 32 direcciones. Los bloques en el último número empiezan en 0, 32, 64, 96, 128… 35 cae en el bloque que empieza en 32.

Red: **130.0.24.32** · Broadcast: **130.0.24.63** (32 + 32 − 1) · Usables: .33 a .62 (30 hosts). El gateway (n24) es .33.

En binario, último número: 35 = 0010 0011, máscara 1110 0000 → AND = 0010 0000 = 32.

Ojo al leer la tabla: "Direcciones" y "Usables" son el tamaño de **una** red de ese prefijo, no cuántas redes hay. La cantidad de redes de cada tipo está en la última columna.

Truco del "número mágico": 256 − (valor de la máscara en el número donde corta) = tamaño del bloque. Para /27 la máscara es 255.255.255.224, y 256 − 224 = 32.

Prefijo	Máscara	Direcciones de cada red	Usables de cada red	Dónde se usó en el TP (cuántas redes así)
/22	255.255.252.0	1024	—	1: el bloque de Net1 (no es una red: es el "barrio" que se sumariza)
/23	255.255.254.0	512	510	1: Net5 (500 hosts)
/24	255.255.255.0	256	254	2 redes: las LANs de Net1 (200 c/u). Además, 5 bloques: Net2, Net3, Net4, Net6 y enlaces leaf–spine
/25	255.255.255.128	128	126	2: Net2 y Net3 (100 c/u)
/26	255.255.255.192	64	62	1: Net3 (60)
/27	255.255.255.224	32	30	3: Net2 (30) y las dos de Net4 (20 c/u)
/28	255.255.255.240	16	14	1: Net6 (10)
/30	255.255.255.252	4	2	**16: cada enlace punto a punto es su propio /30** (2 IPs útiles = una para cada router)

Tipos y direcciones especiales

- **Unicast** (a una interfaz), **broadcast** (a todos los de una red), **multicast** (a un grupo, 224.0.0.0 a 239.255.255.255), **anycast** (al más cercano de un grupo; usa direcciones unicast).
- **Loopback:** 127.0.0.0/8 (127.0.0.1, localhost). Nunca sale a la red.
- **Broadcast limitado:** 255.255.255.255. **"Este host":** 0.0.0.0 (lo usa DHCP antes de tener IP).
- **Privadas (RFC 1918):** 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. No se rutean en Internet (para salir a Internet un router tiene que traducirlas, eso es NAT).

130.0.0.0 es un rango **público** (sería clase B en el esquema de clases viejo). En el TP se usa como si fuera nuestro bloque público asignado.

3.3 Subnetting fijo y VLSM: nuestro reparto

> **Subnetting**
>
> Tomar bits de la parte de host de un bloque para crear subredes. Con n bits prestados salen 2^n subredes. La cátedra permite usar la primera y la última subred (la "subred cero").

> **Subnetting fijo**
>
> Todas las subredes con la misma máscara. Problema: la máscara tiene que alcanzar para la red más grande, y las chicas desperdician direcciones (un enlace de 2 hosts con un /23 sería absurdo).

> **VLSM (Variable Length Subnet Mask, RFC 1009)**
>
> Cada subred tiene la máscara que necesita. Así se aprovecha el bloque.

> **En criollo**
> Tenemos un terreno (el bloque 130.0.16.0/20) y hay que lotearlo para casas de distinto tamaño (las redes). VLSM es darle a cada casa un lote del tamaño justo. Se reparten primero los lotes grandes y después los chicos, para que no queden pedacitos sueltos que no le sirvan a nadie.

Procedimiento de VLSM (el de la cátedra)

1. Ordenar las redes de **mayor a menor** cantidad de hosts.
2. Para cada red, buscar el menor h tal que $2^h - 2 \geq$ hosts pedidos (contando el gateway, que también es una IP). El prefijo es 32 − h.
3. Asignar la primera porción libre de ese tamaño, partiendo el espacio libre en mitades sucesivas. El resto queda libre para la siguiente.
4. Al final, los enlaces punto a punto (/30).

¿Por qué de mayor a menor? Porque cada subred tiene que empezar en un múltiplo de su tamaño (estar **alineada**). Si se asignan primero las chicas, quedan huecos que las grandes no pueden usar.

Paso 1: qué tamaño necesita cada red

Para cada red buscamos la potencia de 2 más chica que alcance para sus hosts, recordando que se pierden 2 direcciones (la de red y la de broadcast). Después, prefijo = 32 − bits de host.

Red	Hosts pedidos	Potencia de 2 que alcanza	Usables ($2^h - 2$)	Bits de host (h)	Prefijo (32 − h)
Net5	500	$512 = 2^9$ (256 no alcanza)	510	9	/23
Net1: LAN de n1 y LAN de n2	200 c/u	$256 = 2^8$	254	8	/24
Net2: LAN de n18 · Net3: LAN de n21	100 c/u	$128 = 2^7$	126	7	/25
Net3: LAN de n22	60	$64 = 2^6$	62	6	/26
Net2: LAN de n17	30	$32 = 2^5$	30	5	/27
Net4: LAN de n23 y LAN de n24	20 c/u	$32 = 2^5$ (16 no alcanza)	30	5	/27
Net6	10	$16 = 2^4$	14	4	/28
Cada enlace entre routers	2	$4 = 2^2$	2	2	/30

¿Cuál LAN de Net2 es la de 30 y cuál la de 100? El dibujo no lo marca. El enunciado dice "una red de 30 host y otra de 100", y el grupo lo tomó en el orden del dibujo, de izquierda a derecha: n17 (izquierda) = 30, n18 (derecha) = 100. Lo mismo en Net3: n21 = 100, n22 = 60.

¿Alcanza el bloque? Sumando los tamaños: 512 + 256 + 256 + 128 + 128 + 64 + 32 + 32 + 32 + 16 + 16 × 4 = **1520** direcciones. El /20 asignado tiene 2^{12} = **4096**. Sobra mucho, y el grupo usó esa sobra para ordenar las redes por Net (paso 2).

Paso 2: un bloque propio para cada Net (direccionamiento jerárquico)

> **La idea, en criollo**
> En vez de poner todas las redes una pegada a la otra, el grupo le dio a cada Net su propio **"barrio"**: un bloque redondo donde entran sus dos LANs y sus dos enlaces con el leaf. Adentro de cada barrio se repartió con VLSM.
> **¿Para qué?** Para que los spines necesiten **un solo cartel por Net**: "todo lo que empiece con 130.0.22 → leaf2". Eso se llama **sumarizar** y se explica en el punto b (sección 5.2).
> **¿El costo?** Quedan direcciones sin usar dentro de cada barrio. No importa: necesitamos 1520 y tenemos 4096.

Primero, cuánto necesita cada barrio. Se suman las redes de esa Net (sus dos LANs y sus dos enlaces /30 con el leaf) y se busca la potencia de 2 que alcance:

Barrio	Qué tiene adentro	Suma	Bloque que alcanza	Prefijo
Net1	2 LANs de 256 + 2 enlaces de 4	520	512 no alcanza (520 > 512) → **1024**	/22
Net5	1 LAN de 512 (cuelga directo de spine1, sin enlaces propios)	512	512	/23
Net2	128 + 32 + 4 + 4	168	256	/24
Net3	128 + 64 + 4 + 4	200	256	/24
Net4	32 + 32 + 4 + 4	72	128 alcanzaba; el grupo reservó **256** (un tercer número entero, más fácil de leer)	/24
Net6	1 LAN de 16 (cuelga directo de spine2)	16	16 (/28); el grupo reservó un /24 entero para el barrio	/24
Enlaces leaf–spine	8 enlaces de 4	32	32 alcanzaba; el grupo reservó un /24	/24

¿Por qué la red de Net1 sola no alcanza con un /23? Porque un /23 (512) se llena exactamente con las dos LANs de 256, y no queda lugar para los dos enlaces /30.

Después, ubicar los barrios en el bloque, de mayor a menor y alineados. Regla de alineación pensando en el tercer número: un /24 ocupa un tercer número entero; un /23 ocupa dos y tiene que empezar en un tercer número **par**; un /22 ocupa cuatro y tiene que empezar en un **múltiplo de 4**.

Orden	Barrio	Tamaño	Desde	Hasta	¿Alineado?
1	Net1	1024 (/22)	130.0.16.0	130.0.19.255	16 es múltiplo de 4 ✓
2	Net5	512 (/23)	130.0.20.0	130.0.21.255	20 es par ✓
3	Net2	256 (/24)	130.0.22.0	130.0.22.255	✓
4	Net3	256 (/24)	130.0.23.0	130.0.23.255	✓
5	Net4	256 (/24)	130.0.24.0	130.0.24.255	✓
6	Net6	256 (/24)	130.0.25.0	130.0.25.255	✓
7	Enlaces leaf–spine	256 (/24)	130.0.26.0	130.0.26.255	✓
—	Libre	1280	130.0.27.0	130.0.31.255	para crecer

Paso 3: adentro de cada barrio, VLSM de mayor a menor

Mismo método de antes: dentro de cada barrio se empieza por la red más grande, cada una arranca donde terminó la anterior (**inicio siguiente = inicio actual + tamaño**) y al final van los dos enlaces /30.

Barrio	Red	Tamaño	Subred	Cuenta
Net1 130.0.16.0/22	LAN de n1 (200)	256	130.0.16.0/24	tercer número 16 entero
	LAN de n2 (200)	256	130.0.17.0/24	+1 en el tercer número
	enlace n1 – leaf1	4	130.0.18.0/30	tercer número 18, de .0 a .3
	enlace n2 – leaf1	4	130.0.18.4/30	0 + 4 = 4 (libre después: 130.0.18.8 a 130.0.19.255)
Net2 130.0.22.0/24	LAN de n18 (100)	128	130.0.22.0/25	0 + 128 = 128
	LAN de n17 (30)	32	130.0.22.128/27	128 + 32 = 160
	enlace n17 – leaf2	4	130.0.22.160/30	160 + 4 = 164
	enlace n18 – leaf2	4	130.0.22.164/30	164 + 4 = 168 (libre .168 a .255)
Net3 130.0.23.0/24	LAN de n21 (100)	128	130.0.23.0/25	0 + 128 = 128
	LAN de n22 (60)	64	130.0.23.128/26	128 + 64 = 192
	enlace n21 – leaf3	4	130.0.23.192/30	192 + 4 = 196
	enlace n22 – leaf3	4	130.0.23.196/30	196 + 4 = 200 (libre .200 a .255)
Net4 130.0.24.0/24	LAN de n23 (20)	32	130.0.24.0/27	0 + 32 = 32
	LAN de n24 (20)	32	130.0.24.32/27	32 + 32 = 64
	enlace n23 – leaf4	4	130.0.24.128/30	el grupo saltó a .128 (dejó .64 a .127 libres para crecer)
	enlace n24 – leaf4	4	130.0.24.132/30	128 + 4 = 132
Net5	LAN (500)	512	130.0.20.0/23	ocupa el barrio entero
Net6	LAN (10)	16	130.0.25.0/28	de .0 a .15 (libre .16 a .255)
Enlaces	8 enlaces leaf–spine	4 c/u	130.0.26.0/30 a 130.0.26.28/30	0, 4, 8 … 28 (libre .32 a .255)

Saltar en Net4 de .64 a .128 es válido: un /30 solo necesita empezar en un múltiplo de 4, y 128 lo es. Si preguntan, es espacio reservado dentro del barrio.

El mismo reparto dibujado como árbol (cada bloque se parte en dos mitades hasta llegar al tamaño justo; es la forma en que lo muestra la cátedra):

```
130.0.16.0/20
 ├─ 130.0.16.0/21
 │   ├─ 130.0.16.0/22  → barrio Net1
 │   │   ├─ 130.0.16.0/24   → LAN n1 (200)
 │   │   ├─ 130.0.17.0/24   → LAN n2 (200)
 │   │   ├─ 130.0.18.0/30 y 130.0.18.4/30 → enlaces n1–leaf1 y n2–leaf1
 │   │   └─ 130.0.18.8 a 130.0.19.255  libre
 │   └─ 130.0.20.0/22
 │       ├─ 130.0.20.0/23  → Net5 (500)
 │       └─ 130.0.22.0/23
 │           ├─ 130.0.22.0/24  → barrio Net2
 │           └─ 130.0.23.0/24  → barrio Net3
 │
 └─ 130.0.24.0/21
     ├─ 130.0.24.0/22
     │   ├─ 130.0.24.0/23
     │   │   ├─ 130.0.24.0/24  → barrio Net4
     │   │   └─ 130.0.25.0/24  → barrio Net6 (usa 130.0.25.0/28)
     │   └─ 130.0.26.0/23
     │       ├─ 130.0.26.0/24  → enlaces leaf–spine (8 × /30)
     │       └─ 130.0.27.0/24  libre
     └─ 130.0.28.0/22  libre
```

Paso 4: dentro de cada red, quién es quién

Convención usada en el TP: la **primera IP usable** es para el router (gateway) y las siguientes para los hosts. Ejemplo en la LAN de n24: red .24.32, gateway n24 = .24.33, n32 = .24.34, n33 = .24.35, broadcast .24.63.

Red	Hosts	Subred	Rango usable	Broadcast	Gateway	Host del TP
Net5	500	130.0.20.0/23	.20.1 – .21.254	.21.255	.20.1 spine1	n34 .20.2
Net1, LAN n1	200	130.0.16.0/24	.16.1 – .16.254	.16.255	.16.1 n1	n11 .16.2
Net1, LAN n2	200	130.0.17.0/24	.17.1 – .17.254	.17.255	.17.1 n2	n12 .17.2
Net2, LAN n18	100	130.0.22.0/25	.22.1 – .22.126	.22.127	.22.1 n18	n14 .22.2
Net2, LAN n17	30	130.0.22.128/27	.22.129 – .22.158	.22.159	.22.129 n17	n13 .22.130
Net3, LAN n21	100	130.0.23.0/25	.23.1 – .23.126	.23.127	.23.1 n21	n29 .23.2
Net3, LAN n22	60	130.0.23.128/26	.23.129 – .23.190	.23.191	.23.129 n22	n30 .23.130
Net4, LAN n23	20	130.0.24.0/27	.24.1 – .24.30	.24.31	.24.1 n23	n31 .24.2
Net4, LAN n24	20	130.0.24.32/27	.24.33 – .24.62	.24.63	.24.33 n24	n32 .24.34, n33 .24.35
Net6	10	130.0.25.0/28	.25.1 – .25.14	.25.15	.25.1 spine2	n35 .25.2

> **Detalle fino: la red de 30 hosts**
> La LAN de n17 pide 30 hosts y un /27 tiene exactamente 30 usables. Si contamos el gateway como una IP más, serían 31 y no entraría. Interpretación usual: "30 hosts" incluye todo lo conectado, o el enunciado ya contempla el router. Si te lo preguntan, decí que con el gateway serían 31 y que lo estricto sería un /26 (130.0.22.128/26 entraba igual en el barrio de Net2, corriendo los enlaces a .192). Lo mismo, con más margen, para la LAN de n22 (60 + 1 = 61 ≤ 62: entra).

3.4 Enlaces punto a punto: /30 y /31

Un enlace punto a punto conecta exactamente dos interfaces (dos routers), sin switch en el medio.

- **/30:** 4 direcciones: red, 2 usables, broadcast. Es lo clásico.
- **/31 (RFC 3021):** 2 direcciones, las dos usables. No hace falta dirección de red ni broadcast en un enlace de solo dos extremos. Ahorra la mitad.

En el TP se usaron **16 enlaces /30**. Los 8 enlaces **acceso–leaf** están adentro del barrio de su Net, y por eso quedan cubiertos por la ruta sumarizada de esa Net (punto b). Los 8 enlaces **leaf–spine** están todos juntos en 130.0.26.0/24.

Enlace	Subred	Extremo A	Extremo B
n1 – leaf1	130.0.18.0/30	n1 .18.1	leaf1 .18.2
n2 – leaf1	130.0.18.4/30	n2 .18.5	leaf1 .18.6
n17 – leaf2	130.0.22.160/30	n17 .22.161	leaf2 .22.162
n18 – leaf2	130.0.22.164/30	n18 .22.165	leaf2 .22.166
n21 – leaf3	130.0.23.192/30	n21 .23.193	leaf3 .23.194
n22 – leaf3	130.0.23.196/30	n22 .23.197	leaf3 .23.198
n23 – leaf4	130.0.24.128/30	n23 .24.129	leaf4 .24.130
n24 – leaf4	130.0.24.132/30	n24 .24.133	leaf4 .24.134
leaf1 – spine1	130.0.26.0/30	leaf1 .26.1	spine1 .26.2
leaf1 – spine2	130.0.26.4/30	leaf1 .26.5	spine2 .26.6
leaf2 – spine1	130.0.26.8/30	leaf2 .26.9	spine1 .26.10
leaf2 – spine2	130.0.26.12/30	leaf2 .26.13	spine2 .26.14
leaf3 – spine1	130.0.26.16/30	leaf3 .26.17	spine1 .26.18
leaf3 – spine2	130.0.26.20/30	leaf3 .26.21	spine2 .26.22
leaf4 – spine1	130.0.26.24/30	leaf4 .26.25	spine1 .26.26
leaf4 – spine2	130.0.26.28/30	leaf4 .26.29	spine2 .26.30

Patrón de los enlaces: el extremo de abajo (el más cercano a los usuarios) tiene la primera IP usable y el de arriba la segunda.

Cómo defenderlo en el coloquio

1. **El bloque:** "Nos asignaron 130.0.16.0/20, que son 4096 direcciones. Necesitábamos 1520, así que sobraba."
2. **El tamaño de cada red:** "Para cada red buscamos el menor $2^h - 2$ que alcance para los hosts. 500 → /23, 200 → /24, 100 → /25, 60 → /26, 30 y 20 → /27, 10 → /28, enlaces → /30." (Mostrar la tabla del paso 1.)
3. **El orden:** "Le dimos a cada Net un bloque propio y alineado, de mayor a menor: Net1 un /22 porque sus dos /24 más los enlaces no entran en un /23, Net5 su /23, y Net2, Net3, Net4 y Net6 un /24 cada una. Los enlaces leaf–spine en 130.0.26.0/24."
4. **El porqué:** "Así los spines tienen una sola ruta por Net. Es direccionamiento jerárquico: se pierde algo de espacio, pero la tabla queda chica y ordenada."
5. **Adentro de cada Net:** "VLSM de mayor a menor; la primera IP de cada LAN es el gateway. Los enlaces acceso–leaf están dentro del bloque de su Net."
6. **Los enlaces:** "/30 porque tienen dos extremos. Con /31 (RFC 3021) se ahorraba la mitad, también era válido."

Qué mostrar en CORE: `ip addr` en un host (n33: 130.0.24.35/27) y en un router (leaf1: cuatro interfaces, cuatro redes). Y calcular en el momento la red y el broadcast de cualquier IP que te pidan, con el número mágico.

Preguntas probables de este punto

1. ¿Qué bloque les asignaron y alcanza?

130.0.16.0/20, 4096 direcciones. Necesitamos 1520 (incluyendo 16 enlaces /30). Alcanza y sobra; por eso pudimos reservar un bloque redondo por Net.

2. ¿Por qué Net5 es /23?

500 hosts + gateway no entran en un /24 (254 usables). Un /23 tiene 512 direcciones, 510 usables.

3. ¿Por qué Net1 ocupa un /22 si son dos redes /24?

Porque además de las dos LANs (2 × 256 = 512) el barrio tiene los dos enlaces /30 con leaf1 (8 direcciones más). 520 no entra en un /23 (512), así que el siguiente bloque alineado es un /22 (1024). Lo que sobra (130.0.18.8 a 130.0.19.255) queda libre.

4. ¿Por qué no pusieron todas las redes pegadas, una atrás de otra?

Se podía, y es válido, pero las redes de un mismo leaf quedan desparramadas y el spine necesita varias rutas por leaf. Con un bloque por Net, el spine tiene una sola ruta por Net (sumarización). El costo es espacio sin usar, y el bloque sobraba.

5. ¿Por qué se asigna de mayor a menor?

Para que cada subred quede alineada a su tamaño, sin huecos. Si se empieza por las chicas, las grandes no caen en un límite válido.

6. Dirección de red, broadcast y rango de Net6.

130.0.25.0/28: red .0, broadcast .15, usables .1 a .14 (14). Gateway spine2 .25.1, n35 .25.2.

7. ¿Por qué /30 en los enlaces? ¿Y /31?

Un enlace punto a punto necesita 2 IPs; un /30 da 2 usables. El /31 (RFC 3021) también sirve y ahorra 2 direcciones por enlace porque no necesita red ni broadcast.

4. Punto a: IPv6 en Net5 y Net6

Qué pide el enunciado: "Configurar la red Net5 y Net6 con IPv6 con el bloque asignado y configurar para que exista conectividad IPv6 entre ambas."

Qué hizo el grupo / qué hay que mostrar: Hecho. Net5 y Net6 tienen un /64 cada una dentro de 2002:dae1::/32. Como spine1 y spine2 no están conectados entre sí, el IPv6 tiene que pasar por un leaf: se eligió **leaf3**, y sus dos enlaces con los spines también tienen IPv6. El resto de la red es solo IPv4. Las rutas IPv6 están en el punto b.

Lo que tenés que poder explicar:

- Cómo se escribe y se abrevia una dirección IPv6.
- Por qué toda LAN IPv6 es /64.
- Cómo se eligieron las subredes dentro del /32 asignado.
- Por qué hubo que poner IPv6 también en leaf3 y en los spines, y no solo en las LANs.

> **En criollo**
> IPv6 es la versión nueva de IP. Hace lo mismo que IPv4 (darle dirección a cada interfaz y rutear paquetes), pero con direcciones mucho más largas, escritas en hexadecimal (números y letras de la a a la f) separadas por ":". En el TP, los equipos del camino Net5 – Net6 tienen las dos versiones a la vez; el resto solo IPv4.

Por qué IPv6 y qué cambia

- Direcciones de **128 bits** ($2^{128} \approx 3{,}4 \times 10^{38}$).
- Header fijo de **40 bytes**, más simple: sin checksum, sin campos de fragmentación, sin opciones. Lo opcional va en **headers de extensión** encadenados con el campo Next Header.
- Campos renombrados: TTL → **Hop Limit**, Protocol → **Next Header**, TOS → **Traffic Class**. Nuevo: **Flow Label** (20 bits).
- No hay broadcast: se usa **multicast**. No hay ARP: se usa **NDP**.
- Fragmentación solo en el origen. Autoconfiguración (SLAAC).

Notación

- 8 grupos de 16 bits en hexadecimal, separados por ":".
- Se pueden omitir los ceros a la izquierda de cada grupo: 0050 → 50.
- Una secuencia de grupos todos en cero se reemplaza por **"::"**, **una sola vez** por dirección.
- No hay máscara decimal: solo largo de prefijo (/64).

Ejemplo del TP: `2002:dae1:0:50::2` = `2002:dae1:0000:0050:0000:0000:0000:0002`. Ojo: ese **50 es hexadecimal** (vale 80 en decimal); se eligió porque se lee "Net 5".

Tipos de direcciones

Tipo	Prefijo	Para qué
Global unicast	2000::/3	Públicas, como las IPv4 públicas. Las del TP (2002:dae1::/32) son de este rango
Link-local	fe80::/10 (se usa fe80::/64)	Solo válidas en el enlace. **Toda interfaz IPv6 tiene una**, aunque no se configure. NDP las usa como origen
Unique Local (ULA)	fc00::/7 (en la práctica fd00::/8)	Equivalente a las privadas de IPv4
Multicast	ff00::/8	ff02::1 todos los nodos del enlace, ff02::2 todos los routers
Solicited-node multicast	ff02::1:ff00:0/104	Para NDP: ff02::1:ff + últimos 24 bits de la dirección
Loopback / no especificada	::1 / ::	localhost / "todavía no tengo dirección"
Documentación	2001:db8::/32	Ejemplos

Regla del /64

En IPv6 **toda LAN es un /64**: 64 bits de red y 64 bits de identificador de interfaz (IID), sin importar cuántos hosts tenga. SLAAC necesita 64 bits de IID para autogenerar direcciones. No se hace VLSM por cantidad de hosts: sobra espacio. Por eso Net5 (500 hosts) y Net6 (10 hosts) tienen el mismo tamaño en IPv6.

Plan del TP

De dónde salen las subredes. El bloque asignado es **2002:dae1::/32**: los dos primeros grupos (32 bits) son fijos. Para llegar a /64 quedan libres el 3.º y el 4.º grupo (64 − 32 = 32 bits). El grupo dejó el 3.º en 0 (es decir, usó el primer /48: **2002:dae1:0::/48**) y numeró las redes con el **4.º grupo**. Cuentas: un /32 contiene 2^{16} = 65.536 /48, y un /48 contiene 65.536 /64.

Subred (/64)	Dónde	Direcciones
2002:dae1:0:**50**::/64	Net5	spine1 ::1 (gateway), n34 ::2
2002:dae1:0:**60**::/64	Net6	spine2 ::1 (gateway), n35 ::2
2002:dae1:0:**f4**::/64	enlace leaf3 – spine1	leaf3 ::1, spine1 ::2
2002:dae1:0:**f5**::/64	enlace leaf3 – spine2	leaf3 ::1, spine2 ::2

Los números del 4.º grupo son solo etiquetas fáciles de leer: 50 = "Net5", 60 = "Net6", f4 y f5 = enlaces. En los enlaces punto a punto también se usó /64 (es lo recomendado por simplicidad; existe la opción /127, RFC 6164, equivalente al /31 de IPv4).

¿Por qué IPv6 también en leaf3 y en los spines?

Un paquete IPv6 solo puede atravesar equipos que hablen IPv6. Net5 cuelga de spine1 y Net6 de spine2, pero **los spines no están conectados entre sí**: el camino más corto entre ellos pasa por un leaf (spine1 → leaf → spine2). Cualquier leaf servía; el grupo eligió **leaf3**. Por eso tienen IPv6: n34, spine1 (en Net5 y

en el enlace con leaf3), leaf3 (en sus dos enlaces con los spines), spine2 (en el enlace con leaf3 y en Net6) y n35.

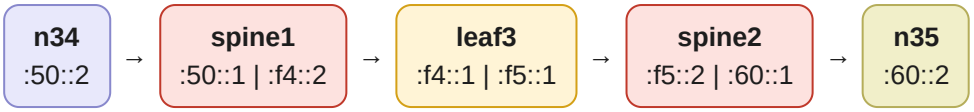


Figura 2. El camino IPv6 de Net5 a Net6. Todas las direcciones empiezan con 2002:dae1:0.

Dual stack

Los equipos de ese camino tienen IPv4 e IPv6 a la vez (doble pila), y las dos funcionan de forma independiente: rutas, ARP/NDP, ping/ping6. Es la técnica de transición más común. Otras: túneles (6in4, GRE, **6to4**) y traducción.

Dato: 2002::/16 es el rango reservado para 6to4 (2002:WWXX:YYZZ:: embebe una IPv4; dae1 = 218.225). Como el bloque lo asignó la cátedra, no es un error; solo conviene saber qué es si preguntan.

Cómo defenderlo en el coloquio

1. **Qué se hizo:** "Le dimos a Net5 la 2002:dae1:0:50::/64 y a Net6 la 2002:dae1:0:60::/64, del bloque 2002:dae1::/32. El gateway de cada una es el spine (::1) y el host es ::2."
2. **Por qué /64:** "En IPv6 toda LAN es /64, sin importar los hosts: 64 bits de red y 64 de identificador de interfaz."
3. **Por qué leaf3:** "Los spines no están conectados entre sí, así que para unir Net5 con Net6 hay que pasar por un leaf. Elegimos leaf3 y le pusimos IPv6 a sus enlaces con los dos spines (:f4:: y :f5::)."
4. **Por qué no en toda la red:** "El punto a pide conectividad entre Net5 y Net6, y con ese camino alcanza. Si hiciera falta en otra red, se le da otro /64 y se agregan sus rutas."

Qué mostrar en CORE: `ip -6 addr` en n34 (se ve la global :50::2 y además una fe80:: link-local) y `ping6 2002:dae1:0:60::2` desde n34.

Preguntas probables de este punto

1. **¿Por qué pusieron IPv6 en leaf3 y en los spines si el punto a pedía solo Net5 y Net6?**
   Porque para que Net5 y Net6 se comuniquen, todos los equipos del camino tienen que rutear IPv6. Los spines no están unidos entre sí, así que el camino es spine1 – leaf3 – spine2.
2. **¿Por qué leaf3 y no otro leaf?**
   Cualquiera servía: todos los leafs están conectados a los dos spines. Se eligió uno; con uno alcanza.
3. **¿Por qué /64 en todas las redes IPv6?**
   Es el estándar para una LAN: 64 bits de identificador de interfaz, que necesita SLAAC. La cantidad de hosts no importa.
4. **¿Qué significa el 50 de 2002:dae1:0:50::?**
   Es el 4.º grupo, el que numera las subredes dentro del /48. Está en hexadecimal (0x50 = 80); se eligió porque se lee "Net5".
5. **¿Qué es una dirección link-local?**
   fe80::/64, válida solo en el enlace, autogenerada en toda interfaz IPv6. NDP la usa como origen.

6. ¿Qué es 2002::/16?

El rango reservado para el mecanismo de transición 6to4. En el TP es simplemente el bloque que asignó la cátedra.

5. Punto b: ruteo estático IPv4 e IPv6

Qué pide el enunciado: "Resolver con ruteo estático la topología para IPv4 e IPv6."

Qué hizo el grupo / qué hay que mostrar: Hecho y verificado (todas las rutas funcionan de ida y vuelta). IPv4: cada host tiene ruta por defecto a su router, cada router de acceso a su leaf; cada leaf conoce sus dos LANs, manda Net6 a spine2 y el resto a spine1; cada spine tiene una ruta sumarizada por Net y no tiene default. IPv6: rutas en spine1, leaf3 y spine2 para unir Net5 con Net6.

Lo que tenés que poder explicar:

- Qué es una tabla de ruteo, una ruta por defecto y el next-hop.
- La regla del prefijo más largo (longest prefix match).
- Qué es sumarizar y dónde se hizo (una ruta por Net en los spines, /60 en IPv6).
- El camino completo de un paquete, de ida y de vuelta.
- Qué pasa si se cae spine1.

5.1 Ruteo y tablas de ruteo

Ruteo (routing)

Decidir el próximo salto (next-hop) y la interfaz de salida para un paquete. Lo hacen hosts y routers, mirando su tabla de ruteo. Es el plano de **control**.

Forwarding (despacho)

Pasar un paquete que entró por una interfaz hacia otra. Solo lo hacen los routers. Es el plano de **datos**. En Linux, un equipo reenvía solo si tiene `net.ipv4.ip_forward = 1`; en CORE eso lo activa el servicio **IPForward**.

En criollo

Cada router es como un cruce de rutas con carteles. Cuando llega un paquete, el router lee la dirección de destino, busca el cartel que coincide y lo manda por esa salida. No conoce el camino completo: solo sabe a qué vecino pasárselo. Si cada router del camino tiene bien sus carteles, el paquete llega.

Tabla de ruteo

Cada entrada tiene: **red destino**, **máscara/prefijo**, **next-hop** (o "directamente conectada"), **interfaz de salida** y **métrica**.

- **Rutas directamente conectadas:** aparecen solas al ponerle IP a una interfaz. El router ya sabe llegar a esa red.
- **Rutas estáticas:** las carga el administrador a mano: `ip route add <red>/<prefijo> via <next-hop>`.
- **Rutas dinámicas:** las aprende un protocolo de ruteo (RIP, OSPF, BGP). No se usaron en este TP (por eso el archivo se llama "estático").
- **Ruta por defecto:** `ip route add default via <next-hop>`. Se usa si nada más coincide.

Regla de oro: longest prefix match

Si varias entradas coinciden con el destino, el router usa la de **prefijo más largo** (la más específica). Ejemplo en leaf1: un paquete a 130.0.25.2 (n35) coincide con `130.0.25.0/24` (vía spine2) y con `default` (vía spine1). Gana /24 → va por spine2.

Regla del next-hop

El next-hop tiene que ser una IP de un **vecino directo**, es decir, estar en una red directamente conectada. Si no, el router no sabe por qué interfaz mandarlo ni a quién hacerle ARP.

Regla de ida y vuelta

Un ping funciona solo si el Echo Request llega **y** el Echo Reply vuelve. Cada router del camino de vuelta también necesita una ruta hacia la red de origen. Muchos errores de los TPs son rutas que funcionan de ida pero no de vuelta.

Host vs router

Un **host** tiene una tabla mínima: su red (conectada) y una default a su gateway. No reenvía paquetes ajenos. Un **router** tiene varias interfaces y reenvía. Un **host multihomed** tiene varias interfaces pero no rutea.

5.2 CIDR y sumarización

CIDR (Classless Inter-Domain Routing, 1993)

Elimina las clases A/B/C: toda red se describe con dirección + prefijo, de cualquier largo. Permite **agregar** (sumarizar, o "supernetting") varias redes en un prefijo más corto. Su objetivo principal es **achicar las tablas de ruteo**.

En criollo

Sumarizar es reemplazar varios carteles por uno solo que los abarca. En vez de que spine1 tenga cuatro carteles para leaf1 (las LANs 130.0.16.x y 130.0.17.x y los enlaces 130.0.18.0 y 130.0.18.4), tiene uno solo: "todo 130.0.16.x a 130.0.19.x → leaf1" (130.0.16.0/22). Se puede hacer solo si las redes están juntas, forman un bloque "redondo" y van al mismo vecino. Por eso en el direccionamiento se le dio a cada Net su propio bloque.

Condiciones para sumarizar varias rutas en una

1. Las redes son **contiguas** y juntas forman un bloque **alineado** (la primera es múltiplo del tamaño total). En la práctica: comparten todos los bits hasta el nuevo prefijo.
2. Todas tienen el **mismo next-hop y la misma interfaz de salida** (regla de la cátedra). Si no, no se sumariza.
3. El sumario no debe cubrir redes que estén en otro lado. Si no, se manda tráfico al lugar equivocado. (Puede cubrir direcciones que no usa nadie: eso no molesta.)

Método: escribir en binario y contar los bits en común

Ejemplo 1, el barrio de Net1. Las redes detrás de leaf1 son 130.0.16.0/24, 130.0.17.0/24, 130.0.18.0/30 y 130.0.18.4/30. Los dos primeros números (130.0) son iguales: 16 bits. Miramos el tercero:

```
16 → 0001 0000
17 → 0001 0001
18 → 0001 0010
      ^^^^ ^^      6 bits iguales en el tercer número
16 + 6 = 22 bits en común  →  sumario 130.0.16.0/22  (cubre 130.0.16.0 a 130.0.19.255)
```

El /22 también cubre 130.0.18.8 a 130.0.19.255, que no usa nadie: no molesta.

Ejemplo 2, el barrio de Net2. 130.0.22.0/25, 130.0.22.128/27, 130.0.22.160/30 y 130.0.22.164/30 tienen los tres primeros números iguales (130.0.22): 24 bits → **130.0.22.0/24**. Igual para Net3 (130.0.23.0/24) y Net4 (130.0.24.0/24).

Dónde se sumarizó en el TP: en los spines, una ruta por Net: `130.0.16.0/22` (Net1), `130.0.22.0/24` (Net2), `130.0.23.0/24` (Net3), `130.0.24.0/24` (Net4). Como los enlaces acceso–leaf están dentro del barrio, quedan cubiertos sin rutas extra. Net6 se anuncia como `130.0.25.0/24` (el barrio entero) aunque la red sea un /28.

Dónde no se pudo: los cuatro enlaces del **otro** spine (por ejemplo, en spine1: 130.0.26.4/30, .12/30, .20/30, .28/30). Son contiguos de a pares, pero cada uno se alcanza por un leaf distinto: **distinto next-hop**, así que van por separado.

> **Ruta por defecto: la sumarización máxima**
> La ruta `default` es 0.0.0.0/0: cubre todo. Se usa cuando todas las demás redes no conectadas salen por el mismo next-hop. La cátedra lo muestra así: si todas las rutas de un router van al mismo vecino, se reemplazan por una default. **Solo puede haber una ruta default por tabla.**

5.3 Ruteo estático en nuestra topología

Qué es un spine y un leaf, en simple: ver sección 1.

> **Spine-leaf**
> Arquitectura de data center en dos niveles. Cada **leaf** se conecta a **todos** los **spines**. Los spines no se conectan entre sí y los leafs tampoco. Así, entre dos leafs cualesquiera siempre hay exactamente un spine en el medio, y hay tantos caminos alternativos como spines.

Lógica por tipo de equipo

Equipo	Rutas cargadas	Por qué
Hosts	default → su gateway	Todo lo que no es su LAN lo resuelve el router
Routers de acceso	default → su leaf	Solo tienen un camino hacia el resto
Leafs	sus 2 LANs → cada router de acceso Net6 (130.0.25.0/24) → spine2 default → spine1	Net6 solo cuelga de spine2. Todo lo demás (Net5, los otros leafs) se alcanza por spine1
Spines	una ruta por Net → el leaf de esa Net (sumarizada) la red del otro spine → vía leaf3 los /30 del otro spine → vía el leaf de ese enlace sin default	El spine es el núcleo y debe conocer todo. Para llegar al otro spine tiene que pasar por un leaf

Las rutas a los /30 del otro spine están comentadas en el archivo como "solo para poder hacer ping a esas interfaces": sin ellas, la red funciona igual entre hosts, pero no se podría hacer ping, por ejemplo, desde Net5 a la IP de spine2 en el enlace con leaf1.

Tablas completas de ejemplo

n11 (host de Net1)

Destino	Next-hop	Interfaz	Origen
130.0.16.0/24	—	eth0	conectada
default	130.0.16.1 (n1)	eth0	estática

n1 (router de acceso)

Destino	Next-hop	Interfaz	Origen
130.0.16.0/24	—	eth0	conectada
130.0.18.0/30	—	eth1	conectada
default	130.0.18.2 (leaf1)	eth1	estática

leaf1

Destino	Next-hop	Interfaz	Origen
130.0.18.0/30, 130.0.18.4/30, 130.0.26.4/30, 130.0.26.0/30	—	eth0, eth1, eth3, eth4	conectadas
130.0.16.0/24 (LAN de n1)	130.0.18.1 (n1)	eth0	estática
130.0.17.0/24 (LAN de n2)	130.0.18.5 (n2)	eth1	estática
130.0.25.0/24 (Net6)	130.0.26.6 (spine2)	eth3	estática
default	130.0.26.2 (spine1)	eth4	estática

leaf2, leaf3 y leaf4 son iguales con sus propias redes:

Leaf	LAN 1	LAN 2	Net6 vía spine2	default vía spine1
leaf2	130.0.22.128/27 → .22.161 (n17)	130.0.22.0/25 → .22.165 (n18)	.26.14	.26.10
leaf3	130.0.23.0/25 → .23.193 (n21)	130.0.23.128/26 → .23.197 (n22)	.26.22	.26.18
leaf4	130.0.24.0/27 → .24.129 (n23)	130.0.24.32/27 → .24.133 (n24)	.26.30	.26.26

spine1

Destino	Next-hop	Qué es
130.0.20.0/23 · 130.0.26.0/30 · .8/30 · .16/30 · .24/30	— (conectadas)	Net5 y sus 4 enlaces con los leafs
130.0.16.0/22	130.0.26.1 (leaf1)	Net1 entera (LANs y enlaces de acceso)
130.0.22.0/24	130.0.26.9 (leaf2)	Net2 entera
130.0.23.0/24	130.0.26.17 (leaf3)	Net3 entera
130.0.24.0/24	130.0.26.25 (leaf4)	Net4 entera
130.0.25.0/24	130.0.26.17 (leaf3)	Net6 (leaf3 la pasa a spine2)
130.0.26.4/30 · .12/30 · .20/30 · .28/30	.26.1 · .26.9 · .26.17 · .26.25	enlaces de spine2 con cada leaf

spine2 es simétrico: 130.0.16.0/22 vía .26.5 (leaf1), 130.0.22.0/24 vía .26.13 (leaf2), 130.0.23.0/24 vía .26.21 (leaf3), 130.0.24.0/24 vía .26.29 (leaf4), **Net5 130.0.20.0/23 vía .26.21 (leaf3)**, y los enlaces de spine1 (.26.0, .8, .16, .24) vía el leaf de cada uno.

Recorridos verificados (ida y vuelta)

- **n11 → n33:** n11 → n1 (default) → leaf1 (default) → spine1 (130.0.24.0/24) → leaf4 (130.0.24.32/27) → n24 → n33. Vuelta: n33 → n24 (default) → leaf4 (default) → spine1 (130.0.16.0/22) → leaf1 (130.0.16.0/24) → n1 → n11. Simétrico.
- **n34 (Net5) → n35 (Net6):** n34 → spine1 (130.0.25.0/24 vía leaf3) → leaf3 (130.0.25.0/24 vía spine2) → spine2 → n35. Vuelta: spine2 (130.0.20.0/23 vía leaf3) → leaf3 (default vía spine1) → spine1 → n34.
- **n11 → n35:** n11 → n1 → leaf1 (130.0.25.0/24 vía spine2) → spine2 → n35. Vuelta: spine2 (130.0.16.0/22 vía leaf1) → leaf1 → n1 → n11.

> **Debilidad del diseño (pregunta probable)**
> La topología tiene caminos redundantes, pero el ruteo estático usa uno solo: casi todo pasa por spine1. Si spine1 se cae, se cae casi toda la red (y Net5 entera, que cuelga de él), y las rutas estáticas **no detectan fallas ni se reconfiguran solas**. Soluciones: ruteo dinámico (OSPF, o BGP en data centers) o ECMP (Equal-Cost Multi-Path: varias rutas iguales para repartir carga, por ejemplo `ip route add <red> nexthop via A nexthop via B`).

5.4 Ruteo IPv6

Mismas reglas que en IPv4 (prefijo más largo, next-hop vecino directo, ida y vuelta), con el comando `ip -6 route add`. Solo hacen falta rutas en los cinco equipos del camino:

Equipo	Ruta	Next-hop	Para qué
n34	default	2002:dae1:0:50::1 (spine1)	todo lo que no es Net5
n35	default	2002:dae1:0:60::1 (spine2)	todo lo que no es Net6
spine1	2002:dae1:0:60::/60	2002:dae1:0:f4::1 (leaf3)	ir a Net6
spine1	2002:dae1:0:f5::/64	2002:dae1:0:f4::1 (leaf3)	enlace leaf3–spine2 (solo para ping)
leaf3	2002:dae1:0:50::/60	2002:dae1:0:f4::2 (spine1)	ir a Net5
leaf3	2002:dae1:0:60::/60	2002:dae1:0:f5::2 (spine2)	ir a Net6
spine2	2002:dae1:0:50::/60	2002:dae1:0:f5::1 (leaf3)	ir a Net5
spine2	2002:dae1:0:f4::/64	2002:dae1:0:f5::1 (leaf3)	enlace leaf3–spine1 (solo para ping)

¿Por qué /60 si las redes son /64?

El 4.º grupo de Net5 es `0050`, que en binario es `0000 0000 0101 0000`. Un /60 son 48 + 12 bits: fija los primeros 12 bits del 4.º grupo (`0000 0000 0101`) y deja libres los últimos 4. Entonces `2002:dae1:0:50::/60` cubre de `:50::` a `:5f::`: 16 redes /64. Es una ruta que ya deja lugar para que Net5 crezca a 16 subredes sin tocar las tablas. Con /64 hubiera funcionado igual hoy.

```
/sbin/ip -6 route add 2002:dae1:0:60::/60 via 2002:dae1:0:f4::1    # en spine1: Net6 por leaf3
```

Cómo defenderlo en el coloquio

1. **La lógica, de abajo hacia arriba:** "Los hosts tienen default a su gateway; los routers de acceso, default a su leaf, porque tienen una sola salida."
2. **Leafs:** "Cada leaf conoce sus dos LANs por los routers de acceso. Net6 la manda a spine2, que es donde está conectada, y todo lo demás por default a spine1."
3. **Spines:** "Los spines son el núcleo: tienen una ruta por Net, sumarizada gracias a que cada Net tiene su bloque (130.0.16.0/22, 130.0.22.0/24, 130.0.23.0/24, 130.0.24.0/24). No tienen default porque conocen todo. Para llegar a la red del otro spine pasan por leaf3."
4. **Reglas que se cumplen:** "Todos los next-hops son vecinos directos y verificamos que cada camino funciona de ida y de vuelta."
5. **IPv6:** "Lo mismo con ip -6 route, solo en el camino n34 – spine1 – leaf3 – spine2 – n35. En los spines y leaf3 usamos /60 por Net."
6. **Lo que reconocés antes de que te lo marquen:** "Con estático casi todo pasa por spine1; si se cae, no hay reruteo. En un caso real se usaría OSPF/BGP o ECMP."

Qué mostrar en CORE: `ip route` en leaf1 y en spine1 (se ven las rutas de las tablas de arriba), `ip -6 route` en leaf3, y `ip route get 130.0.24.35` en leaf1 para mostrar qué ruta elige.

Preguntas probables de este punto

1. **¿Qué es una topología spine-leaf?**
   Dos niveles: cada leaf conectado a todos los spines; ni spines ni leafs entre sí. Entre dos leafs siempre hay un solo spine en el medio y caminos redundantes.
2. **¿Qué es la ruta por defecto y cuántas puede haber?**
   0.0.0.0/0, el prefijo más corto; se usa si ninguna otra coincide. Una sola por tabla.
3. **¿Cómo elige el router entre varias rutas?**
   Longest prefix match: gana la más específica. Ejemplo: en leaf1, para ir a 130.0.25.2 gana 130.0.25.0/24 (vía spine2) sobre la default (vía spine1).
4. **¿Qué es sumarizar y dónde lo hicieron?**
   Representar varias redes contiguas con un prefijo más corto para achicar la tabla. En los spines: una ruta por Net (130.0.16.0/22 para Net1 y un /24 para Net2, Net3, Net4 y Net6), y /60 en IPv6.
5. **¿Por qué Net1 se sumariza con /22?**
   Sus redes (16.0/24, 17.0/24, 18.0/30, 18.4/30) comparten 22 bits: en binario, 16, 17 y 18 comparten los primeros 6 bits del tercer número (16 + 6 = 22).
6. **¿Por qué los spines no tienen ruta por defecto?**
   Porque son el núcleo y conocen todas las redes. Una default no tendría a dónde apuntar: no hay nada "afuera".
7. **¿Por qué los leafs mandan Net6 a spine2 y el resto a spine1?**
   Net6 solo está conectada a spine2. Para todo lo demás alcanza una default a spine1, que conoce toda la red.
8. **¿Cómo llega spine1 a Net6 si no está conectado a spine2?**
   Vía leaf3, que tiene ruta a Net6 por spine2. Spine1 → leaf3 → spine2 → Net6.
9. **¿Qué pasa si se cae spine1?**
   Se pierde casi toda la conectividad, porque las rutas estáticas no se adaptan. Se resuelve con ruteo dinámico (OSPF/BGP) o ECMP.
10. **¿Qué hace el servicio IPForward?**
    Activa el reenvío de paquetes (ip_forward en IPv4 y forwarding en IPv6) para que el nodo actúe como router.

6. Punto d: ping y traceroute

Qué pide el enunciado: "Realizar test con ping (ICMP) y traceroute para probar que funciona la topología."

Qué hizo el grupo / qué hay que mostrar: Hay que ejecutarlo en CORE (no queda guardado en el archivo): pings entre hosts de distintas redes y un traceroute. Si todo responde, el direccionamiento y el ruteo están bien.

Lo que tenés que poder explicar:

- Qué es ICMP y qué mensajes usa ping (Echo Request 8 / Echo Reply 0).
- Cómo funciona traceroute (TTL creciente, Time Exceeded, Port Unreachable).
- Qué saltos debería mostrar el traceroute de n11 a n33.

> **ICMP (RFC 792)**
> Protocolo "helper" de IP para reportar errores y hacer consultas. Va encapsulado dentro de IP (Protocol = 1), pero no es de transporte. Es informativo: no corrige nada ni agrega confiabilidad.

Reglas: un mensaje de error ICMP incluye el header IP del datagrama que lo causó más al menos 8 bytes de sus datos. No se genera un error ICMP por otro error ICMP ni por paquetes a multicast.

Mensaje	Tipo	Código	Cuándo aparece
Echo Reply	0	0	Respuesta de ping
Destination Unreachable – Net Unreachable	3	0	El router no tiene ruta a esa red
Destination Unreachable – Host Unreachable	3	1	La red existe pero el host no responde (falla el ARP)
Destination Unreachable – Port Unreachable	3	3	Llegó al host pero el puerto UDP está cerrado (final del traceroute)
Echo Request	8	0	ping
Time Exceeded – TTL en tránsito	11	0	El TTL llegó a 0 en un router (traceroute)

ping (punto d)

Manda Echo Request (8) y espera Echo Reply (0). Mide el RTT y muestra el TTL con que volvió la respuesta (útil para estimar saltos: Linux arranca en 64).

traceroute (puntos d y f)

> **En criollo**
> traceroute sirve para ver **por qué routers pasa** un paquete hasta llegar al destino. El truco: manda paquetes que "mueren" a propósito después de 1 salto, después de 2, después de 3… Cada router donde muere un paquete avisa "se murió acá", y así se va armando la lista de routers del camino.

1. Manda un paquete con TTL = 1. El primer router lo descarta y devuelve **Time Exceeded (11/0)**: así se conoce su IP.

2. Repite con TTL = 2, 3, … Cada vez se revela un router más.
3. En Linux, traceroute manda **UDP** a puertos altos (desde 33434), que nadie usa. Cuando llega al destino, este responde **Port Unreachable (3/3)** y traceroute sabe que terminó.
4. Por defecto manda 3 sondas por salto (los 3 tiempos que muestra). `traceroute -I` usa Echo Request en vez de UDP; `-n` evita resolver nombres.

Salida esperada de `traceroute -n 130.0.24.35` desde n11 (deducida de las tablas; verificarla al correrla):

```
1  130.0.16.1     n1
2  130.0.18.2     leaf1
3  130.0.26.2     spine1
4  130.0.26.25    leaf4
5  130.0.24.133   n24
6  130.0.24.35    n33
```

La IP que aparece en cada salto es la de la interfaz del router que mira hacia n11, porque es la que usa como origen para mandar el ICMP de vuelta.

Pings para mostrar que todo anda

Conviene elegir pares que crucen caminos distintos:

Desde	Hacia	Qué prueba
n11 (Net1)	n33 130.0.24.35 (Net4)	leaf → spine1 → otro leaf (el camino típico)
n11 (Net1)	n35 130.0.25.2 (Net6)	la ruta de los leafs hacia Net6 por spine2
n13 (Net2)	n34 130.0.20.2 (Net5)	la default de los leafs hacia spine1
n34 (Net5)	n35 130.0.25.2 (Net6)	spine1 → leaf3 → spine2 (los dos spines)
n12 (Net1)	n11 130.0.16.2 (Net1)	dos LANs del mismo leaf

Cómo defenderlo en el coloquio

1. **Qué es:** "ping usa ICMP: manda Echo Request (tipo 8) y espera Echo Reply (tipo 0). Si vuelve, hay ruta de ida y de vuelta."
2. **Qué probamos:** "Pings entre redes de distintos leafs, hacia Net5 y Net6 y entre Net5 y Net6, así pasamos por los dos spines." (Mostrar la tabla de pings.)
3. **traceroute:** "Manda sondas con TTL 1, 2, 3…; cada router donde el TTL llega a 0 responde Time Exceeded (11) y así aparece en la lista. El destino responde Port Unreachable (3/3) porque las sondas son UDP a puertos que nadie usa."
4. **Lectura del resultado:** "De n11 a n33 hay 6 líneas: n1, leaf1, spine1, leaf4, n24 y n33. Coincide con las tablas de ruteo."

Qué mostrar en CORE: `ping -c 3 130.0.24.35` y `traceroute -n 130.0.24.35` desde n11. Si no hay CORE en el coloquio, llevar captura de pantalla de cada uno.

Preguntas probables de este punto

1. ¿Cómo funciona traceroute?

Envía sondas con TTL creciente. Cada router responde ICMP Time Exceeded (11/0) y el destino ICMP Port Unreachable (3/3), porque en Linux las sondas son UDP.

2. ¿Qué tipos ICMP vieron?

Echo Request 8, Echo Reply 0, Time Exceeded 11, Destination Unreachable 3 (código 3: puerto inalcanzable).

3. ¿Para qué sirve el TTL?

Evita que un paquete circule para siempre si hay un loop de ruteo. Traceroute lo aprovecha.

7. Punto e: captura n11 → n33 (ARP e ICMP)

Qué pide el enunciado: "Capturar puntualmente el tráfico de n11 hacia n33 y analizar: ARP e ICMP."

Qué hizo el grupo / qué hay que mostrar: Hay que ejecutarlo (no queda guardado en el archivo): vaciar la caché ARP, capturar con tcpdump o Wireshark en el enlace de n11 y hacer ping a n33 (130.0.24.35).

Lo que tenés que poder explicar:

- Qué es ARP y para qué sirve (IP → MAC).
- Por qué n11 hace ARP por su gateway (130.0.16.1) y no por n33.
- Qué cambia y qué no en el paquete en cada salto (MAC sí, IP no).
- Qué mensajes aparecen en la captura y en qué orden.

ARP (RFC 826)

Protocolo "helper" de IP que traduce una **dirección IP** (lógica, 32 bits) a una **dirección MAC** (física, 48 bits) dentro de la misma red local. Se encapsula directo en la trama Ethernet (no va dentro de IP). Funciona solo, sin configuración.

Regla fundamental

ARP solo funciona **dentro de un dominio de broadcast** (la misma LAN o el mismo enlace). No se puede averiguar con ARP la MAC de un equipo de otra red: el router no reenvía broadcasts.

En criollo

Para mandar algo por el cable, el equipo necesita la MAC (la "dirección física") del vecino, pero solo conoce su IP. ARP es gritar en la red local: "¿quién tiene la IP 130.0.16.1? Decime tu MAC". El dueño de esa IP contesta solo a quien preguntó. Después se anota la respuesta para no volver a preguntar.

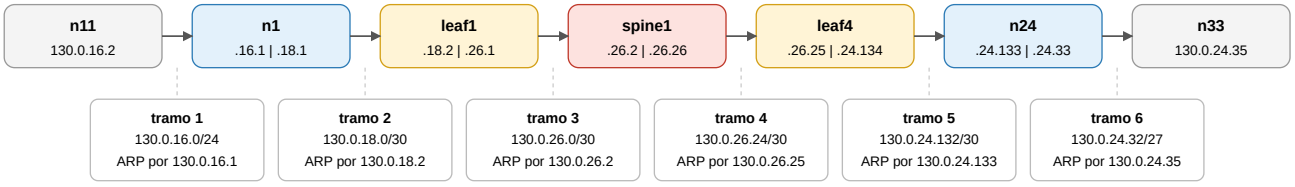
Funcionamiento

1. El emisor quiere mandar un paquete IP y necesita la MAC destino para armar la trama.
2. Mira su **tabla ARP** (caché). Si está la entrada, la usa.
3. Si no está, manda un **ARP Request** en **broadcast** (MAC destino ff:ff:ff:ff:ff:ff): "¿Quién tiene la IP X? Respondele a la IP Y, MAC Z".
4. Todos lo reciben, pero solo responde el que tiene la IP X, con un **ARP Reply unicast**: "X está en la MAC W".
5. El emisor guarda la entrada en la caché (con un tiempo de vida) y manda la trama.

¿Por qué IP pregunta? Los dos casos

- **Destino en mi misma red** (lo decide el emisor con su máscara): ARP por la IP del destino.
- **Destino en otra red**: ARP por la IP de mi **gateway** (el next-hop de la tabla de ruteo). La trama va a la MAC del router, aunque la IP destino del paquete sea la del destino final.

n11 → n33 paso a paso (punto e)



En cada tramo la trama Ethernet es nueva (MAC origen y destino cambian); el paquete IP conserva 130.0.16.2 → 130.0.24.35 y solo baja el TTL.
Las IPs de cada router son: interfaz de entrada | interfaz de salida (ida). traceroute muestra la de la izquierda (la que mira hacia n11).

Figura 3. Camino de n11 a n33: por quién se hace ARP en cada tramo.

1. n11 quiere hacer ping a 130.0.24.35. AND con su máscara: 130.0.24.35 no está en 130.0.16.0/24, así que el paquete va a su default gateway 130.0.16.1.
2. n11 manda un ARP Request broadcast "¿quién tiene 130.0.16.1?". n1 responde con su MAC.
3. n11 manda la trama: MAC origen n11, MAC destino n1 · IP origen 130.0.16.2, IP destino 130.0.24.35 · ICMP Echo Request.
4. n1 recibe, mira su tabla (default vía 130.0.18.2), baja el TTL, recalcula el checksum, hace ARP por 130.0.18.2 (si no lo tiene) y arma una trama nueva hacia leaf1.
5. Lo mismo en cada salto, hasta que n24 ve que 130.0.24.35 está en su red conectada y hace ARP por la IP de n33.

Tramo	MAC origen	MAC destino	IP origen	IP destino
n11 → n1	n11 eth0	n1 eth0	130.0.16.2	130.0.24.35
n1 → leaf1	n1 eth1	leaf1 eth0	130.0.16.2	130.0.24.35
leaf1 → spine1	leaf1 eth4	spine1 eth1	130.0.16.2	130.0.24.35
spine1 → leaf4	spine1 eth4	leaf4 eth3	130.0.16.2	130.0.24.35
leaf4 → n24	leaf4 eth1	n24 eth1	130.0.16.2	130.0.24.35
n24 → n33	n24 eth0	n33 eth0	130.0.16.2	130.0.24.35

Conclusión: las MAC cambian en cada salto (son locales a cada enlace); las IP de origen y destino no cambian (son de extremo a extremo).

Qué se ve en la captura en el enlace de n11: ARP Request (broadcast) → ARP Reply (unicast) → Echo Request → Echo Reply. Si la caché ya estaba cargada no hay ARP: hay que vaciarla antes con `ip neigh flush all`.

Cómo defenderlo en el coloquio

1. **Qué es ARP:** "Traduce IP a MAC dentro de la misma red. El Request va en broadcast y el Reply en unicast; la respuesta se guarda en la caché ARP."
2. **Por quién pregunta n11:** "n11 hace el AND con su máscara, ve que 130.0.24.35 no es de su red 130.0.16.0/24 y entonces hace ARP por su gateway, 130.0.16.1, no por n33."
3. **Qué se ve en la captura:** "ARP Request de n11, ARP Reply de n1, ICMP Echo Request de 130.0.16.2 a 130.0.24.35 y el Echo Reply de vuelta."
4. **Lo importante:** "En cada salto el router arma una trama nueva: cambian las MAC y baja el TTL, pero las IP de origen y destino son las mismas de punta a punta. Cada router hace su propio ARP en su tramo; el ARP por n33 lo hace n24."

Qué mostrar en CORE: en n11, `ip neigh flush all`; en otra terminal de n11, `tcpdump -i eth0 -n -e arp or icmp`; después `ping -c 2 130.0.24.35`. Con `-e` se ven las MAC: la MAC destino del Echo Request es la de n1, no la de n33.

Variantes de ARP (teoría de la clase)

- **Proxy ARP:** un nodo responde un ARP Request en nombre de otro.
- **Gratuitous ARP:** ARP Reply en broadcast sin que nadie lo pida, para anunciar o actualizar el mapeo IP–MAC propio.
- **ARP Probe / Announcement:** al asignarse una IP, el nodo pregunta si alguien la usa (probe); si nadie responde, la reclama (announcement).

Preguntas probables de este punto

1. Si n11 le hace ping a n33, ¿por quién pregunta ARP?
Por 130.0.16.1, su gateway, porque n33 está en otra red. El ARP por 130.0.24.35 lo hace n24.

2. ¿Qué cambia en el paquete en cada salto?
MAC origen y destino (trama nueva), TTL (−1) y checksum del header IP. Las IPs no cambian.

3. ¿Qué se ve en la captura, en orden?
ARP Request de n11 (broadcast, "¿quién tiene 130.0.16.1?"), ARP Reply de n1 (unicast, con su MAC), ICMP Echo Request de 130.0.16.2 a 130.0.24.35 y el Echo Reply de vuelta.

4. ¿Por qué el ARP Request va a broadcast y el Reply no?
El que pregunta no sabe la MAC de nadie, así que pregunta a todos (ff:ff:ff:ff:ff:ff). El que responde ya sabe la MAC del que preguntó (venía en el Request), así que le contesta solo a él.

8. Punto f: traceroute capturado

Qué pide el enunciado: "Realizar un traceroute entre los mismos equipos (n11 → n33), capturar los mensajes."

Qué hizo el grupo / qué hay que mostrar: Hay que ejecutarlo (no queda guardado en el archivo): capturar en el enlace de n11 mientras se corre traceroute -n 130.0.24.35. La teoría de traceroute está en el punto d (sección 6).

Lo que tenés que poder explicar:

- Qué paquetes manda traceroute y con qué TTL.
- Qué mensaje ICMP devuelve cada router y cuál devuelve el destino.

Captura del traceroute (punto f)

En el enlace de n11, en orden:

Sonda de n11	Quién responde	Respuesta
UDP con TTL 1	n1, desde 130.0.16.1	ICMP Time Exceeded (11/0)
UDP con TTL 2	leaf1, desde 130.0.18.2	ICMP Time Exceeded (11/0)
UDP con TTL 3	spine1, desde 130.0.26.2	ICMP Time Exceeded (11/0)
UDP con TTL 4	leaf4, desde 130.0.26.25	ICMP Time Exceeded (11/0)
UDP con TTL 5	n24, desde 130.0.24.133	ICMP Time Exceeded (11/0)
UDP con TTL 6	n33, desde 130.0.24.35	ICMP Port Unreachable (3/3)

Cada fila aparece tres veces (tres sondas por salto, cada una a un puerto UDP distinto desde 33434). Si la caché estaba vacía, al principio se ve además el ARP por el gateway 130.0.16.1.

Cómo defenderlo en el coloquio

1. **Qué manda n11:** "Paquetes UDP a puertos altos con TTL 1, 2, 3… Tres por cada TTL."
2. **Qué devuelven los routers:** "Cada router baja el TTL; al que le llega a 0 descarta el paquete y avisa con ICMP Time Exceeded, tipo 11 código 0. La IP de origen de ese aviso es la de la interfaz que mira hacia n11; por eso vemos 130.0.16.1, 130.0.18.2, 130.0.26.2, 130.0.26.25 y 130.0.24.133."
3. **Qué devuelve n33:** "A n33 la sonda le llega con TTL válido; como el puerto UDP está cerrado, responde ICMP Port Unreachable, tipo 3 código 3, y traceroute sabe que terminó."

Qué mostrar en CORE: en n11, `tcpdump -i eth0 -n udp or icmp` en una terminal y `traceroute -n 130.0.24.35` en otra (o capturar con Wireshark sobre el enlace n11 – switch).

Preguntas probables de este punto

1. ¿Qué mensajes se ven en la captura del traceroute?

Sondas UDP con TTL 1, 2, 3… desde n11. Por cada una, un ICMP Time Exceeded (tipo 11) de cada router del camino. Al final, ICMP Port Unreachable (tipo 3, código 3) desde n33. Al principio puede

aparecer el ARP por el gateway.

2. **¿Por qué el destino responde distinto que los routers?**
   Porque a n33 la sonda le llega con TTL todavía válido, así que la procesa: es UDP a un puerto cerrado y responde Port Unreachable. A los routers les llega con TTL 1, lo bajan a 0, la descartan y avisan con Time Exceeded.
3. **¿Cuántos saltos hay de n11 a n33?**
   Cinco routers (n1, leaf1, spine1, leaf4, n24) más el destino: 6 líneas en el traceroute.
4. **¿Por qué el router responde con la IP de esa interfaz y no con otra?**
   Porque el ICMP lo genera el router y sale por la interfaz que va hacia n11; Linux usa como origen la IP de esa interfaz. Como en nuestra red la ida y la vuelta son por el mismo camino, coincide con la interfaz por donde entró la sonda.

9. Punto h: IPv6 e ICMPv6

Qué pide el enunciado: "Probar conectividad en las redes con IPv6 (por separado), capturar tráfico y analizar ICMPv6."

Qué hizo el grupo / qué hay que mostrar: Hay que ejecutarlo (no queda guardado en el archivo): ping6 y traceroute6 de n34 (Net5) a n35 (Net6) capturando el tráfico. Son los únicos hosts con IPv6.

Lo que tenés que poder explicar:

- Qué es ICMPv6 y por qué es más importante que ICMP en IPv4.
- Que en IPv6 no hay ARP: lo reemplaza NDP (Neighbor Solicitation y Neighbor Advertisement).
- A qué dirección va el Neighbor Solicitation (multicast solicited-node).
- Los tipos ICMPv6 que aparecen: 128/129 (ping6), 135/136 (NDP), 3 (traceroute6).

ICMPv6 (RFC 4443)

Mucho más importante que ICMP en IPv4: además de errores y ping, transporta **NDP** (que reemplaza a ARP y descubre routers) y **MLD** (que reemplaza a IGMP). Si se bloquea ICMPv6, IPv6 deja de funcionar.

NDP – Neighbor Discovery Protocol (RFC 4861)

Mapea direcciones IPv6 a MAC (como ARP), descubre routers y prefijos, detecta direcciones duplicadas (DAD) y vecinos inalcanzables (NUD). Usa 5 mensajes ICMPv6.

Mensaje ICMPv6	Tipo	Uso
Destination Unreachable	1	Sin ruta, puerto cerrado (código 4, al final del traceroute6)
Packet Too Big	2	Paquete mayor que la MTU (PMTU)
Time Exceeded	3	Hop Limit en 0 (traceroute6)
Echo Request / Reply	128 / 129	ping6
Router Solicitation (RS)	133	Un host pide a los routers que se anuncien
Router Advertisement (RA)	134	El router anuncia prefijo y gateway (SLAAC)
Neighbor Solicitation (NS)	135	"¿Quién tiene esta IPv6?" (equivale al ARP Request). También para DAD
Neighbor Advertisement (NA)	136	"La tengo yo, mi MAC es…" (equivale al ARP Reply)
Redirect	137	Avisa de un next-hop mejor

NS / NA vs ARP: la diferencia clave

El NS **no va a broadcast** (IPv6 no tiene). Va a la **solicited-node multicast** de la dirección buscada: `ff02::1:ff` + los últimos 24 bits. Solo escuchan esa dirección los nodos cuyas IPs terminan igual, así que se molesta a muchos menos equipos que con un broadcast.

> **Ejemplo del TP**
> n34 quiere llegar a su gateway 2002:dae1:0:50::1. Últimos 24 bits: 00:0001 → NS a **ff02::1:ff00:1** (en Ethernet, MAC destino 33:33:ff:00:00:01). spine1 responde con un NA unicast con su MAC.

SLAAC (para saber, no se usó)

El router manda RAs con el prefijo /64; el host arma su dirección con prefijo + IID, hace DAD y toma al router como gateway. Los flags del RA indican si además se usa DHCPv6: M = 1 (direcciones por DHCPv6), O = 1 (otros datos, como DNS, por DHCPv6). En el TP todo es estático, por eso no aparecen RS/RA en las capturas.

Qué hacer y qué se ve (punto h)

```
# desde n34 (Net5) hacia n35 (Net6)
ip -6 neigh flush all
ping6 -c 3 2002:dae1:0:60::2
traceroute6 -n 2002:dae1:0:60::2
ip -6 neigh                       # caché de vecinos
tcpdump -i eth0 -n icmp6          # captura
```

En la captura: NS de n34 a ff02::1:ff00:1 → NA de spine1 → Echo Request (128) → Echo Reply (129). Es posible ver también NS/NA usando direcciones **link-local** (fe80::…) como origen.

traceroute6 esperado (deducido de las tablas; verificarlo al correrlo):

```
 1  2002:dae1:0:50::1   spine1
 2  2002:dae1:0:f4::1   leaf3
 3  2002:dae1:0:f5::2   spine2
 4  2002:dae1:0:60::2   n35
```

Los tres primeros responden ICMPv6 Time Exceeded (tipo 3) y n35 responde Destination Unreachable, código 4 (puerto inalcanzable).

Cómo defenderlo en el coloquio

1. **Qué probamos:** "Conectividad IPv6 por separado de IPv4: ping6 y traceroute6 de n34 (Net5) a n35 (Net6), que son los hosts con IPv6."
2. **ICMPv6:** "En IPv6, ICMPv6 es obligatorio: además de ping y errores, lleva NDP. Si se bloquea, IPv6 no anda."
3. **NDP en lugar de ARP:** "n34 no hace ARP: manda un Neighbor Solicitation (135) a la multicast solicited-node de su gateway, ff02::1:ff00:1, y spine1 contesta con un Neighbor Advertisement (136) con su MAC. No hay broadcast en IPv6."
4. **El resto de la captura:** "Echo Request 128 y Echo Reply 129 para ping6; Time Exceeded tipo 3 en cada salto del traceroute6 (spine1, leaf3, spine2). No hay Router Advertisements porque todo es estático."

Qué mostrar en CORE: en n34, `ip -6 neigh flush all`, `tcpdump -i eth0 -n icmp6` en una terminal y `ping6 -c 3 2002:dae1:0:60::2` en otra; después `traceroute6 -n 2002:dae1:0:60::2`.

Preguntas probables de este punto

1. **¿Qué reemplaza a ARP en IPv6?**
   NDP: Neighbor Solicitation (135) a la multicast solicited-node y Neighbor Advertisement (136) unicast.
2. **¿A qué dirección manda n34 el Neighbor Solicitation para encontrar a su gateway?**
   A la multicast solicited-node de 2002:dae1:0:50::1, que es ff02::1:ff00:1 (ff02::1:ff + los últimos 24 bits de la dirección buscada).
3. **¿Por qué no aparecen Router Advertisements en la captura?**
   Porque todo se configuró a mano (estático). Los RA los mandan los routers cuando se usa autoconfiguración (SLAAC), y acá no se activó.
4. **¿Por qué el ping6 solo se puede hacer entre n34 y n35?**
   Porque el punto a pedía IPv6 en Net5 y Net6, y solo esos hosts (y los routers del camino) tienen IPv6. Un host de Net1, por ejemplo, no tiene dirección IPv6 global.

10. CORE y comandos

CORE (Common Open Research Emulator) emula redes en Linux: cada nodo es un contenedor liviano con su propia pila de red. La versión final del TP está guardada en el formato **XML** de CORE (`tpi_1_nuevo.xml`); la versión vieja usaba el formato `.imn`. Se abre desde la interfaz de CORE (File → Open) y se arranca con Start; doble click en un nodo abre su terminal.

Cómo leer el XML

El archivo tiene cuatro partes:

```
<networks>  … los switches (type="SWITCH"), por ejemplo n3, n4, n15 …
<devices>
  <device id="23" name="n1" type="router">      ← id interno y nombre real
    <services> IPForward, StaticRoute </services>  ← rutea y carga su script de rutas
  <device id="2" name="n11" type="host">       ← los hosts usan DefaultRoute
<links>
  <link node1="25" node2="23">                 ← cable entre leaf1 (25) y n1 (23)
    <iface1 name="eth0" ip4="130.0.18.2" ip4_mask="30"/>   ← lado de leaf1
    <iface2 name="eth1" ip4="130.0.18.1" ip4_mask="30"/>   ← lado de n1
<service_configurations>
  <service name="StaticRoute" node="23">  … staticroute.sh de n1:
      /sbin/ip route add default via 130.0.18.2
```

Para saber la IP de un equipo hay que buscar su `id` en los `link`: la interfaz `iface1` es del `node1` y la `iface2` del `node2`. Si un lado es un switch, ese lado no tiene interfaz. Las rutas de cada equipo están al final, en `service_configurations`, con el número de nodo. Los que tienen IPv6 lo muestran con `ip6="2002:dae1:…" ip6_mask="64"`.

id	Nombre	id	Nombre	id	Nombre
1	n34	22	n5-spine1	29	n21
2	n11	23	n1	30	n22
3 / 4 / 5	n12 / n13 / n14	24	n2	31	n9-leaf3
6 / 7	n29 / n30	25	n7-leaf1	32	n23
8 / 9 / 10	n31 / n32 / n33	26 / 27	n17 / n18	33	n24
11	n35	28	n8-leaf2	34 / 35	n10-leaf4 / n6-spine2

Comandos

Comando	Para qué
`ip addr` / `ip -6 addr`	Ver IPs de las interfaces
`ip route` / `ip -6 route`	Ver la tabla de ruteo
`ip route add 130.0.16.0/22 via 130.0.26.1`	Agregar ruta estática
`ip route add default via 130.0.16.1`	Agregar ruta por defecto
`ip route get 130.0.24.35`	Qué ruta usaría para ese destino
`ip neigh` / `ip neigh flush all`	Ver / vaciar la caché ARP (y NDP con -6)
`sysctl net.ipv4.ip_forward`	1 = el nodo rutea
`ping`, `ping6`, `traceroute`, `traceroute6`	Pruebas de conectividad
`tcpdump -i eth0 -n arp or icmp`	Capturar ARP e ICMP
`tcpdump -i eth0 -n -e`	Mostrar también las MAC (útil para el punto e)
`tcpdump -i eth0 -n -w captura.pcap`	Guardar la captura para abrir en Wireshark

11. Errores conceptuales que hay que evitar

- **"El host hace ARP por la IP del destino."** Solo si está en su misma red. Si no, hace ARP por su gateway.
- **"Las IP cambian en cada salto."** No: cambian las MAC. Las IP son de extremo a extremo.
- **"ARP en IPv6 usa broadcast."** No existe ARP ni broadcast en IPv6: NDP con multicast solicited-node.
- **"Un /24 tiene 256 hosts."** Tiene 256 direcciones y 254 usables.
- **"Para que haya ping alcanza con la ruta de ida."** También hace falta la de vuelta.
- **"El next-hop puede ser cualquier router del camino."** Tiene que ser un vecino directo.
- **"Sumarizar es juntar redes cualesquiera."** Tienen que ser contiguas, alineadas y con el mismo next-hop.
- **"En IPv6 hago VLSM según la cantidad de hosts."** No: toda LAN es /64.
- **"El 50 de 2002:dae1:0:50:: es cincuenta."** Es hexadecimal (0x50 = 80). Se eligió porque se lee "Net5".
- **"Un /22 es una red con 1022 hosts."** En el TP el /22 de Net1 no es una LAN: es el bloque que agrupa sus redes para sumarizar.
- **"traceroute usa ICMP Echo."** En Linux usa UDP por defecto; en Windows (tracert), ICMP Echo.